

Enter Spring boot

1. What is Spring Boot?

Spring Boot is a Framework/project/module from "The Spring Team"(spring.io) to simplify the bootstrapping and development of new Spring Applications.

It's built on top of the Spring framework to simplify the development of Spring-based applications

Why spring boot ?

If you are using only spring framework w/o Spring boot.

1. Programmer has to manage n resolve the dependencies in pom.xml

1.1 You have to know exactly which dependencies are necessary n their exact compatible versions.

eg : refer to spring MVC hibernate's pom.xml

1.2 When you compare it with Spring boot ,

it provides dependency management through spring boot starter dependencies.

eg : spring web starter dependency, spring jpa starter, spring security starter....

These pre-configured dependencies to simplify the management of third-party libraries and ensure compatibility between different components of the Spring ecosystem.

So much lesser no of dependencies n NO need to specify their versions !

2. With Spring boot, the configuration part of the application reduces to ZERO.

It supplies automatic configuration support .

Meaning :

Spring Boot looks at

2.1 Frameworks available on the CLASSPATH

2.2 Existing configuration for the application.

Based on these, Spring Boot provides basic configuration needed to configure the application with these frameworks. This is called Auto Configuration.

eg :In ORM configuration(Hikari - DBCP vendor) ,

After selecting Spring Web starter (through spring starter project), it automatically supplies default Spring MVC configuration

(i.e DispatcherServlet, HandlerMapping, ViewResolver)

3. No need to deploy spring boot application on any external web server.

You don't even create a WAR file n deploy it on the external server.

Since Spring boot provides embedded Tomcat server n you can simply create a JAR file which can be deployed on the embedded Tomcat server.

4. Spring boot also provides production ready features such as monitoring , auditing , health checks using additional spring boot dependency : Spring boot Actuator starter.

You can monitor your app , under different environments.

Details :

Key Features summary

Opinionated Defaults: Spring Boot provides a set of opinionated defaults and conventions that enable developers to quickly create production-ready applications with minimal configuration. It automates many common configuration tasks, reducing boilerplate , configuration allowing developers to focus on application logic n to write Unit Tests and Integration Tests easily.

Auto-configuration: Spring Boot's powerful auto-configuration feature analyzes the project's classpath and automatically configures beans and components based on the dependencies detected. It eliminates the need for explicit configuration and reduces the time spent on setting up and wiring components manually.

Embedded Server: Spring Boot includes an embedded server, such as Tomcat or Jetty, allowing applications to be packaged as standalone JAR files. This feature simplifies deployment and makes it easy to run Spring Boot applications without requiring a separate web/application server installation.

Production-Ready Features: Spring Boot offers a range of production-ready features, such as health checks, auditing, monitoring, logging, and external configuration. These features promote best practices and facilitate building robust and monitorable applications.

Dependency Management: Spring Boot provides dependency management through spring boot starter dependencies. These pre-configured dependencies simplify the management of third-party libraries and ensure compatibility between different components of the Spring ecosystem.

Developer Productivity: Spring Boot's emphasis on convention over configuration, automatic configuration, and built-in development tools boosts developer productivity. It reduces the time spent on repetitive setup tasks, allowing developers to focus on business logic and application functionality.

Community Support: Spring Boot has a large and active community, which provides extensive documentation, tutorials, and support.

Microservices and Cloud-Native Applications: Spring Boot is well-suited for building microservices and cloud-native applications. It integrates smoothly with other Spring Cloud projects, providing features for service discovery, distributed configuration, circuit breakers, and more. Spring Boot's lightweight and modular nature works well with the principles of microservices and cloud-native architectures.

2. What is NOT Spring Boot?

Spring Boot Framework is not implemented from the scratch by The Spring Team

It's implemented on top of existing Spring Framework (Spring IO Platform : start.spring.io).

It is not used for solving any new problems. It is used to solve same problems like Spring Framework. (i.e to help in writing enterprise applications)

3. Advantages of Spring Boot:

It is very easy to develop Spring Based applications with Java

It reduces lots of development time and increases productivity.

It avoids writing lots of boilerplate Code, Annotations and XML Configuration.

It is very easy to integrate Spring Boot Application with its Spring Ecosystem like Spring JDBC, Spring ORM, Spring Data, Spring Security etc.

It follows "Opinionated Defaults Configuration" Approach to reduce Developer effort

It provides Embedded HTTP servers like Tomcat, Jetty etc. to develop and test our web applications very easily.

It provides CLI (Command Line Interface) tool to develop and test Spring Boot(Java or Groovy) Applications from command prompt very easily and quickly.

It provides lots of plugins to develop and test Spring Boot Applications very easily using Build Tools like Maven and Gradle

It provides lots of plugins to work with embedded and in-memory Databases very easily.

In short

Spring Boot = Existing Spring Framework + Embedded HTTP Server(eg Tomcat) - XML Based configuration - efforts in identifying dependencies in pom.xml

4. What is that "Opinionated Defaults Configuration" ?

When we use Hibernate/JPA, we would need to configure a datasource, a session factory, a transaction manager among lot of other things.

Refer to our hibernate-persistence.xml , to recall what we did earlier .

Spring Boot says can we look at it differently ?

Can we auto-configure a Data Source(connection pool) / session factory / Tx manager if Hibernate jar is on the classpath?

It says :

When a spring mvc jar is added into an application, can we auto configure some beans automatically? (eg HandlerMapping , ViewResolver n configure DispatcherServlet)

By the way :

There would be of course provisions to override the default auto configuration.

5. How does it work ?

Spring Boot looks at

1. Frameworks available on the CLASSPATH

2. Existing configuration for the application.

Based on these, Spring Boot provides basic configuration needed to configure the application with these frameworks. This is called Auto Configuration.

6. What is Spring Boot Starter ?

Starters are a set of convenient dependency descriptors that you can include in your application's pom.xml

.

eg : Suppose you want to develop a web application.

W/o Spring boot , we would need to identify the frameworks we want to use, which versions of frameworks to use and how to connect them together.

BUT all web application have similar needs.

These include Spring MVC, Jackson Databind (for data binding), Hibernate-Validator (for server side validation using Java Validation API) and Log4j (for logging). Earlier while creating any web app, we had to choose the compatible versions of all these frameworks.

With Spring boot : You just add Spring Boot Starter Web.

Dependency for Spring Boot Starter Web

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Just by adding above starter , it will add lot of JARs under maven dependencies

Another eg : If you want to use Spring and JPA for database access, just include the spring-boot-starter-data-jpa dependency in your project, and you are good to go.

7. Another cool feature of Spring boot is : we don't have to worry about deploying our applications to external container. It comes with an embedded servlet container.

8. Important components of a Spring Boot Application

Below is the starting point of a Spring Boot Application

```
@SpringBootApplication
public class HelloSpringBootApplication {

    public static void main(String[] args) {
        SpringApplication.run(HelloSpringBootApplication.class, args);
    }

}
```

About : org.springframework.boot.SpringApplication

It's a Class used to bootstrap and launch a Spring application from a Java main method.

By default class will perform the following steps to bootstrap the application

1. Create an ApplicationContext instance (representing SC)
2. Manages life cycle of spring beans

@SpringBootApplication - This is where all the spring boot magic happens.
It consists of following 3 annotations.

1. @SpringBootConfiguration

It tells spring boot that this class here can have several bean definitions. We can define various spring beans here and those beans will be available at run time .
(using @Bean annotations)

2. @EnableAutoConfiguration

It tells spring boot to automatically configure the spring application based on the dependencies that it sees on the classpath.

eg:

If we have a MySql dependency in our pom.xml , Spring Boot will automatically create a data source, using the properties in application.properties file.

If we have spring web starter , in pom.xml , then spring boot will automatically create the dispatcher servlet n other beans (HandlerMapping , ViewResolver, DispatcherServlet)

All the xml, all the java based configuration is now gone. It all comes for free thanks to spring boot to enable auto configuration annotation.

3. @ComponentScan (equivalent to xml tag : context:component-scan)

So this tells us that spring boot to scan through the classes and see which all classes are marked with the stereotype annotations like @Component Or @Service @Repository and manage these spring beans . Default base-pkg is the pkg in which main class is defined.
Can be overridden by

eg :

@ComponentScan(basePackages = "com")

For scanning entities : (equivalent to packagesToScan)

@EntityScan(basePackages = "com.app.pojos")